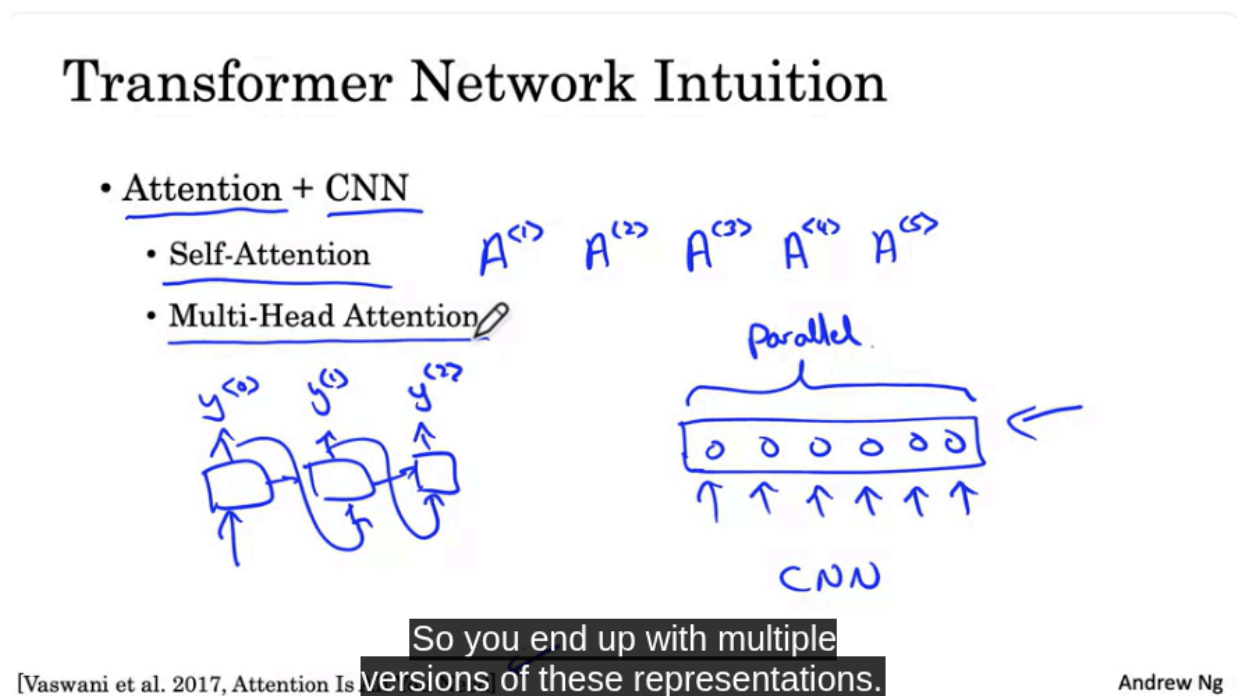
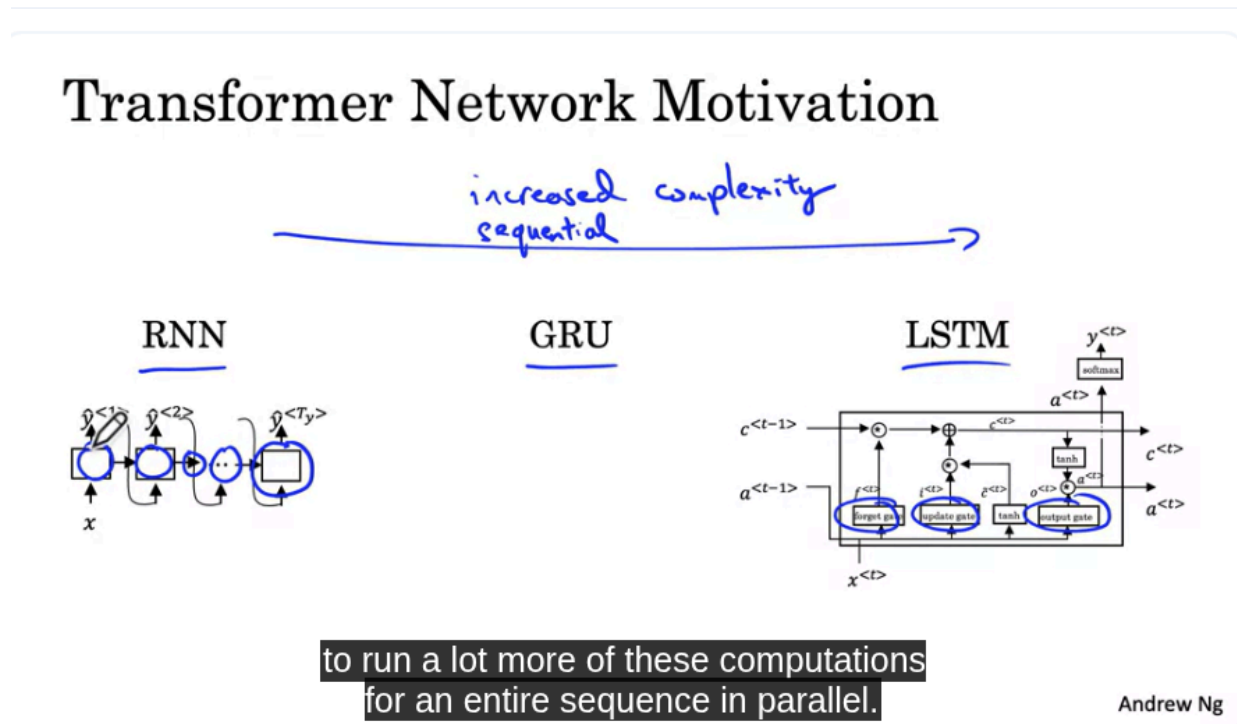


Transformer Network

Transformer Network Intuition



Introduction to Transformers

- The **Transformer** is one of the most important architectures in modern deep learning, especially for **Natural Language Processing (NLP)**.
- Many highly effective NLP systems are based on the Transformer architecture.
- Compared with RNN-based models, Transformers allow much more computation to happen **in parallel**.

Evolution from RNN to Transformer

Sequence models became progressively more sophisticated:

RNN $\rightarrow$ GRU $\rightarrow$ LSTM $\rightarrow$ Transformer

RNN

- Processes a sequence **one token at a time**.
- Suffers from problems such as **vanishing gradients**.

- Has difficulty learning long-range dependencies.

GRU and LSTM

- Introduce **gates** to control information flow.
- Help preserve information across longer sequences.
- Reduce some of the problems of basic RNNs.
- However, they still process sequences sequentially.

Limitation of Sequential Processing

In an RNN-like model:

$$h^{\langle 1 \rangle} \rightarrow h^{\langle 2 \rangle} \rightarrow h^{\langle 3 \rangle} \rightarrow \dots$$

To compute a later state, the model must first compute all earlier states.

For example: $h^{\langle 5 \rangle}$

cannot be computed until:

$$h^{\langle 1 \rangle}, h^{\langle 2 \rangle}, h^{\langle 3 \rangle}, h^{\langle 4 \rangle}$$

have already been computed.

This creates a **sequential bottleneck**.

Main Advantage of Transformers

Transformers can process many tokens in a sequence **at the same time**.

Instead of: $x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4$

the model can process: $[x_1, x_2, x_3, x_4]$

in parallel.

This makes Transformers much more suitable for modern hardware such as GPUs.

Transformer vs. RNN

RNN	Transformer
Sequential processing	Highly parallel processing
One token at a time	Entire sequence can be processed together
Long dependency paths	Direct attention between distant tokens
Harder to parallelize	Highly parallelizable
Uses recurrence	Primarily uses attention

Main Innovation

The Transformer combines two important ideas:

- **Attention-based representations**
- **Parallel processing** similar in spirit to CNN-style computation

Attention allows each word to directly gather useful information from other words in the sequence.

At the same time, these representations can be computed largely in parallel.

Self-Attention

The first major component of the Transformer is **Self-Attention**.

Suppose a sentence contains five tokens: x_1, x_2, x_3, x_4, x_5

Self-attention computes contextualized representations: A_1, A_2, A_3, A_4, A_5

for all five tokens.

Each representation can incorporate information from **other words in the same sentence**.

For example, the representation: A_3

for the third word may depend on: x_1, x_2, x_3, x_4, x_5

rather than only the previous words.

Why Self-Attention Is Powerful

A word's meaning often depends on surrounding words.

For example:

| The **bank** approved the loan.

versus:

| He sat on the **bank** of the river.

Self-attention allows the representation of **bank** to incorporate the surrounding context, producing a richer contextual representation.

Parallel Self-Attention

Unlike an RNN, which computes hidden states sequentially, self-attention can compute:

$$A_1, A_2, A_3, A_4, A_5$$

largely in parallel.

This provides both:

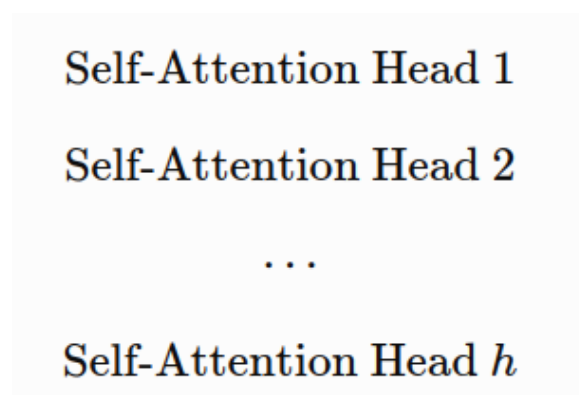
- Rich contextual representations
 - Efficient computation
-

Multi-Head Attention

The next important idea is **Multi-Head Attention**.

Instead of performing self-attention only once, the model performs several attention operations in parallel.

Conceptually:



Each head can learn to focus on different relationships between words.

For example, different heads might capture:

- Syntactic relationships
- Semantic similarity
- Subject-verb relationships

- Long-range dependencies

Self-Attention vs. Multi-Head Attention

Self-Attention

Computes one contextual representation for each token.

$$x_i \rightarrow A_i$$

Self-Attention

Self-Attention Intuition

$A(q, K, V)$ = attention-based vector representation of a word
↳ calculate for each word $A^{(1)}, \dots, A^{(5)}$

RNN Attention

$$\alpha^{<t,t'>} = \frac{\exp(e^{<t,t'>})}{\sum_{t'=1}^T x \exp(e^{<t,t'>})}$$

↓ ↓ ↓

$x^{<1>}$ $x^{<2>}$ $x^{<3>}$

Jane visite l'Afrique

Transformers Attention

$$A(q, K, V) = \sum_i \frac{\exp(q \cdot k^{<i>})}{\sum_j \exp(q \cdot k^{<j>})} v^{<i>}$$

↓ ↓ ↓

$x^{<4>}$ $x^{<5>}$

en septembre

$q^{<3>}$ $k^{<3>}$ $v^{<3>}$

[Vaswani et al. 2017, Attention Is All You Need] Andrew Ng

Self-Attention

$$A(q, K, V) = \sum_i \frac{\exp(q \cdot k^{<i>})}{\sum_j \exp(q \cdot k^{<j>})} v^{<i>}$$

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$A^{<1>}$ $A^{<2>}$ $A^{<3>}$ $A^{<4>}$ $A^{<5>}$

↑ ↑ ↑ ↑ ↑

$v^{<1>}$ $v^{<2>}$ $v^{<3>}$ $v^{<4>}$ $v^{<5>}$

↑ ↑ ↑ ↑ ↑

$q^{<1>} \cdot k^{<1>}$ $q^{<2>} \cdot k^{<2>}$ $q^{<3>} \cdot k^{<3>}$ $q^{<4>} \cdot k^{<4>}$ $q^{<5>} \cdot k^{<5>}$

↑ ↑ ↑ ↑ ↑

$x^{<1>}$ $x^{<2>}$ $x^{<3>}$ $x^{<4>}$ $x^{<5>}$

Jane visite l'Afrique en septembre

Query (Q) Key (K) Value (V)

$q^{<1>}$	$k^{<1>}$ person	$v^{<1>}$
$q^{<2>}$	$k^{<2>}$ action	$v^{<2>}$
$q^{<3>}$ What's	$k^{<3>}$	$v^{<3>}$
$q^{<4>}$ happens	$k^{<4>}$	$v^{<4>}$
$q^{<5>}$ there	$k^{<5>}$	$v^{<5>}$

$q^{<3>} = W^Q \cdot x^{<3>}$
 $k^{<3>} = W^K \cdot x^{<3>}$
 $v^{<3>} = W^V \cdot x^{<3>}$

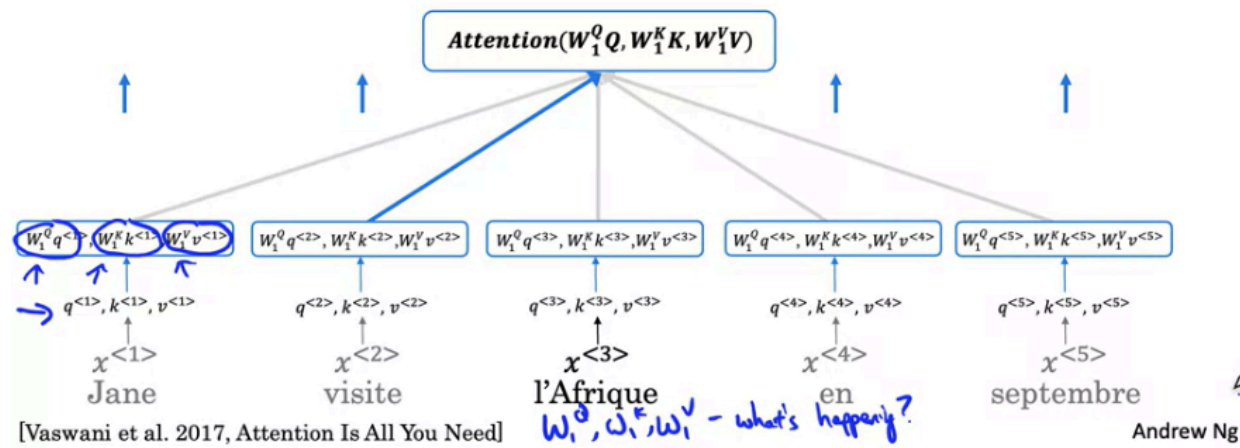
[Vaswani et al. 2017, Attention Is All You Need] Andrew Ng

Multi-Head Attention

Multi-Head Attention

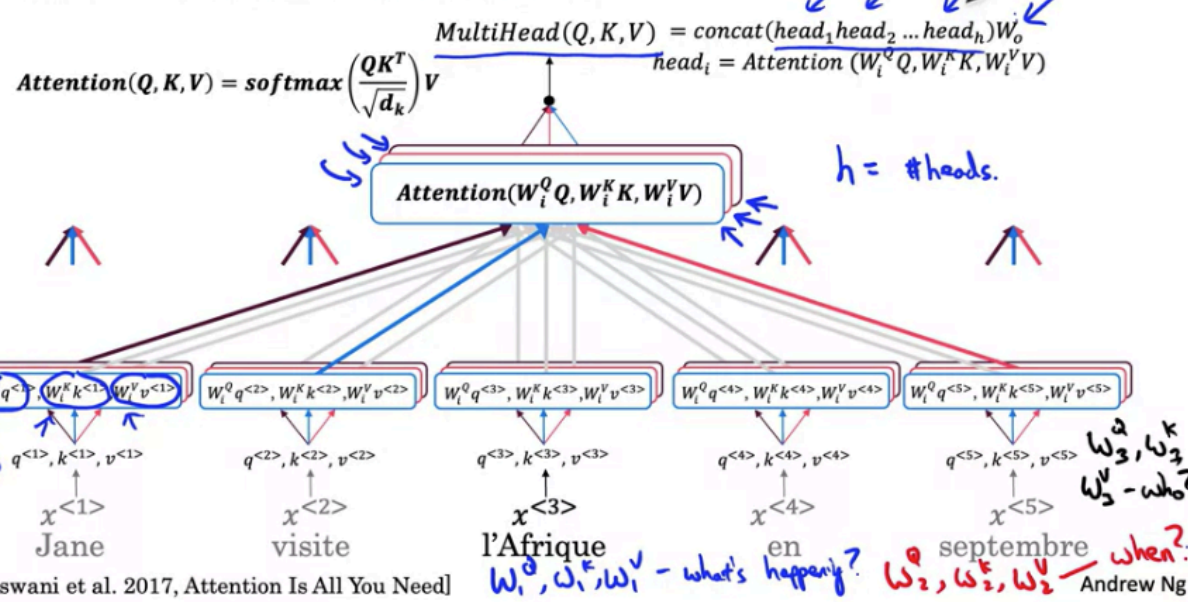
MultiHead(Q, K, V)

"head"



Multi-Head Attention

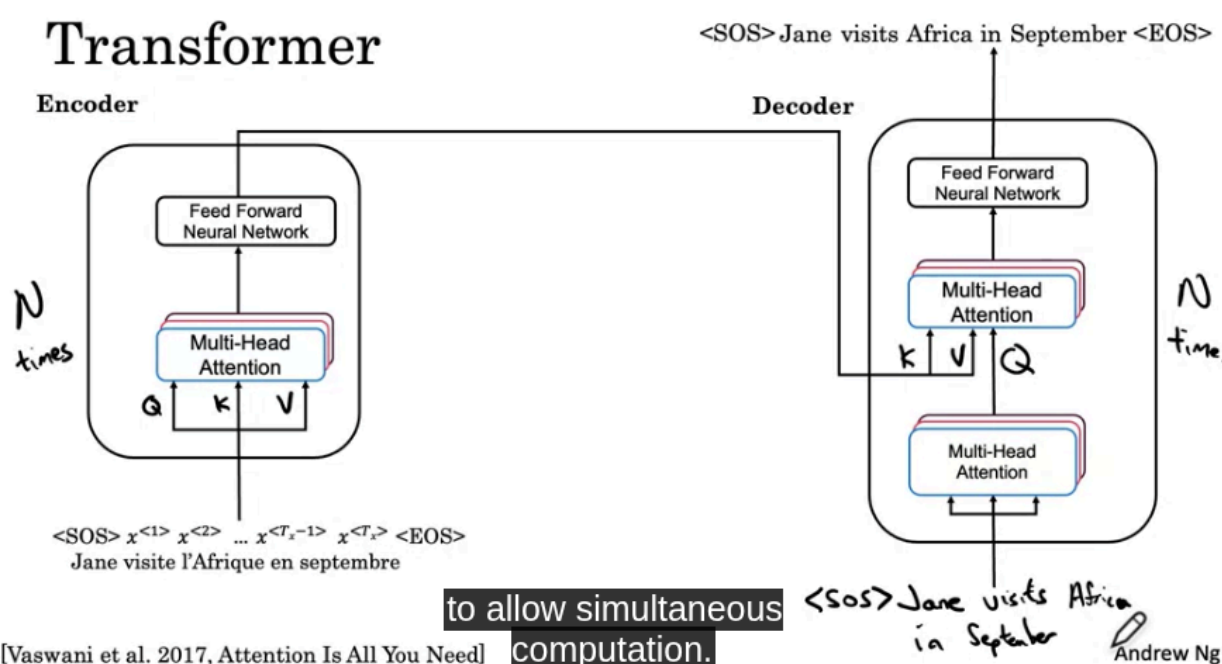
"head"



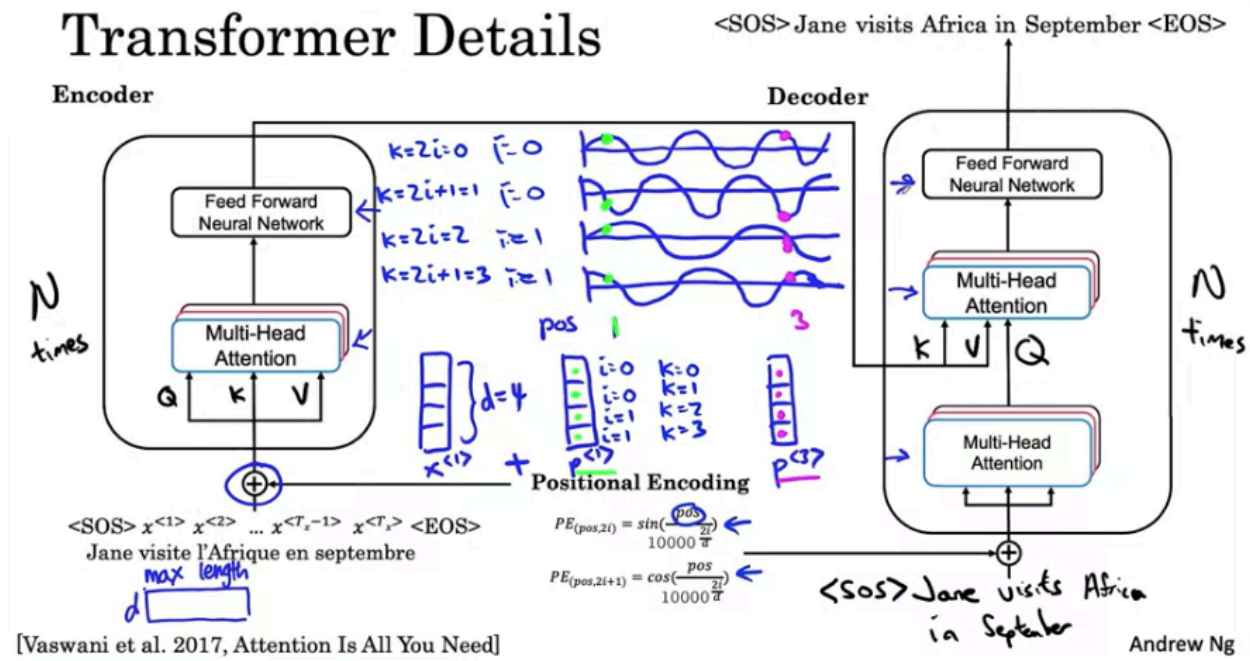
We could compute the heads parallelly

Transformer Network

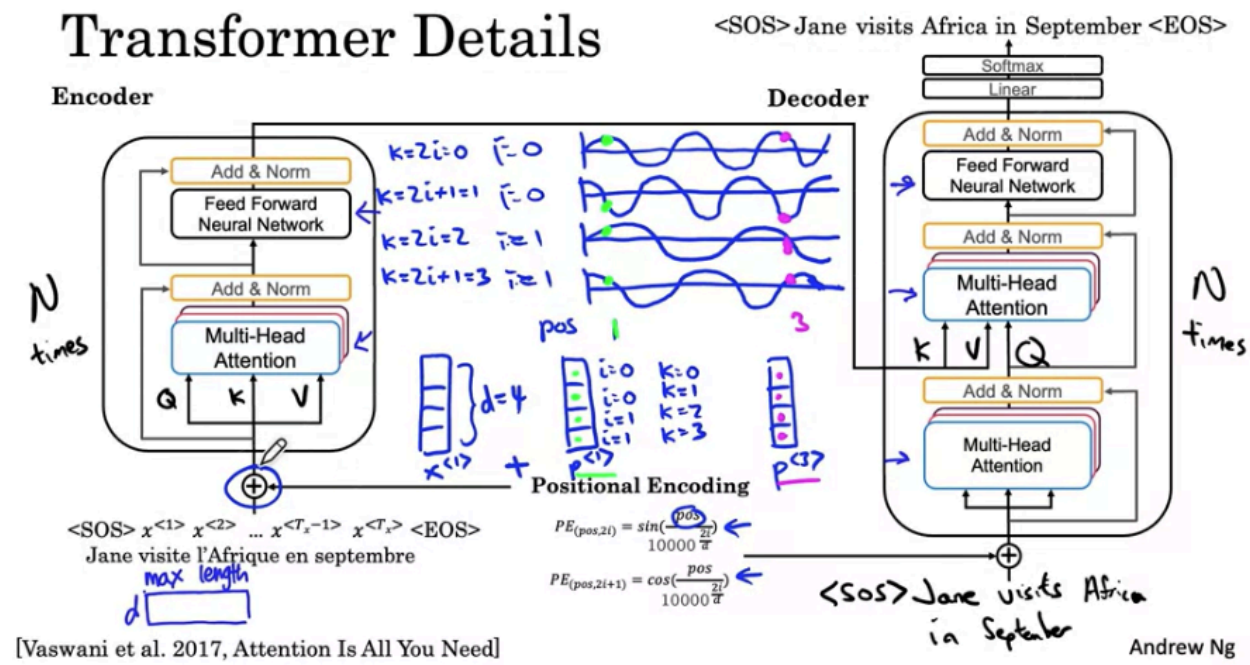
Transformer



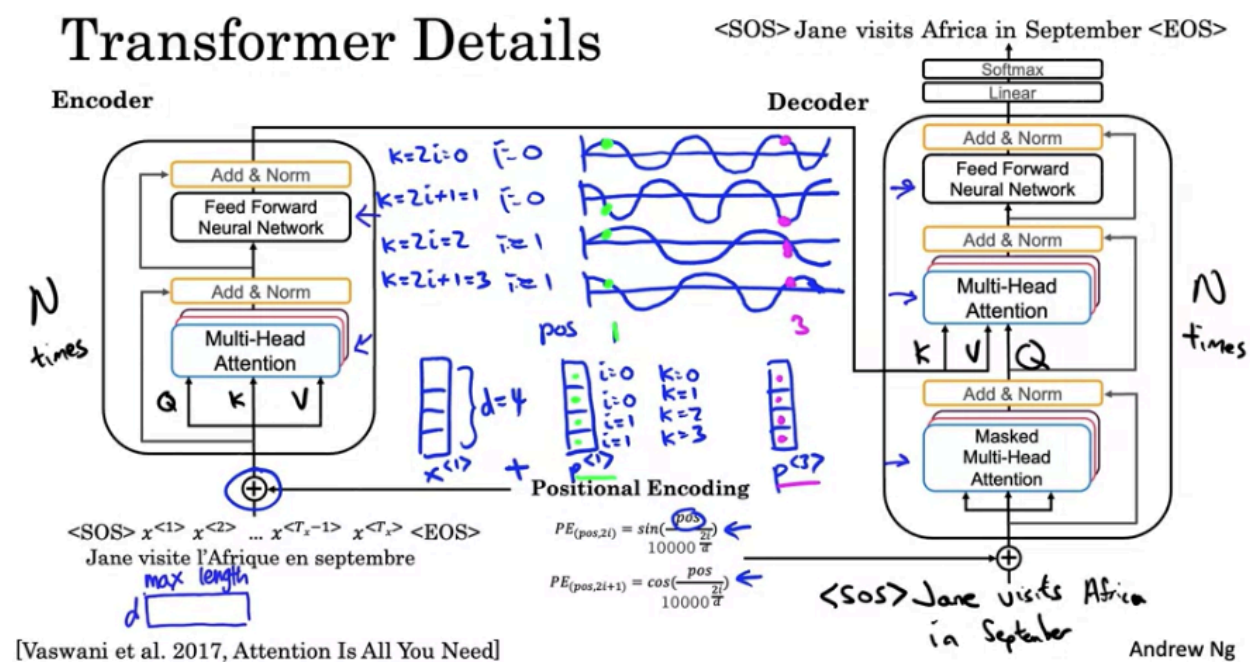
Transformer Details



Transformer Details



Transformer Details



Transformer Details

